

Jonathan Brugh

Enhancement Three Narrative

08/09/2026

Overview

Enhancement Three represents a fundamentally different kind of change from the previous two revisions. Enhancement One hardened the console program around its data structure, and Enhancement Two corrected and completed the binary search tree (BST) itself, but in both versions the course catalog still lived and died with the process, meaning every course was held in memory as a tree of Node objects, and closing the program erased it. Enhancement Three removes the BST entirely and replaces it with a MongoDB-backed CourseRepository class built on the official mongocxx driver. The course catalog is now a persistent collection of BSON documents that exists independently of any single run of the program, and the CRUD operations that used to walk a tree in memory now issue queries, indexes, and atomic writes against that collection. This is the enhancement most clearly aimed at demonstrating database competency: schema design in a document model, index-enforced data integrity, atomic upsert semantics, bulk-write behavior, and defensive handling of a data store that, unlike an in-memory tree, can be unreachable, slow, or already holding conflicting data.

Replacing an In-Memory Structure With a Persistent Collection

The most visible database concept this enhancement demonstrates is persistence itself. In Enhancement One and Two, the entire program's data lived inside root, the BST's single in-memory entry point. The moment main() returned, that memory, and every course in it, was gone. Enhancement Three's main() opens a mongocxx::client against a MongoDB URI and

hands a specific collection, `client["snhu_courses"]["courses"]`, to `CourseRepository`. From that point on, every insert, update, and delete operates on a collection that lives in the database server's own storage, not in the program's process memory. A user can load a CSV, exit the program, relaunch it, and immediately query the same courses without reloading anything, which is something no earlier version of this application could do, because none of them had anywhere for the data to live once the program stopped running. This could have been rectified by reading and writing to the same, or new, CSV file in the earlier revisions, but that was not the intent of either of the previous enhancements. Moving toward a database management system was the clear evolutionary future of this program; however, this progression brings its own new challenges. `Main()` demonstrates an awareness that a database, unlike a data structure inside your own process, is a separate system that might not be available. Before doing any real work, it runs a ping command against the database and reports the outcome, an explicit connectivity check that has no equivalent in the BST versions because there was never anything external to check.

[Moving a Correctness Guarantee Into the Database Itself](#)

Enhancement Two enforced `courseId` uniqueness at the application layer. `addNode()` checked for a matching key before inserting a new tree node, so the guarantee only held as long as every insertion path in that specific program went through that specific function. Enhancement Three moves the same guarantee into the database with `CreateUniqueIndex()`, which builds a unique index on the `courseId` field directly on the MongoDB collection. This is a meaningful conceptual shift, where uniqueness is no longer something the C++ code has to remember to check as it is a constraint the database itself will reject a violation of, for any write from any client, present or future. This is exactly the kind of database design decision a software engineer is expected to

internalize: that data integrity rules belong as close to the data as possible, not scattered across every piece of application code that happens to touch it.

Atomic Upsert Instead of a Read-Then-Write Pattern

Enhancement Two's add-or-update workflow in the menu is a two-step, application-level pattern: call `Search()` to see if a course exists, then call either `Insert()` or `UpdateCourse()` based on the result. That pattern works for a single-user console program, but it is not safe in general, because nothing stops the data from changing between the read and the write. Enhancement Three's `UpsertCourse()` replaces this with a single call to `replace_one()` using `mongocxx::options::replace` with `upsert(true)` creating one atomic database operation that inserts the document if no course with that ID exists, or replaces it if one does, and reports which case occurred through the result's `upserted_id()` and `modified_count()`. Collapsing a read-check-then-write sequence into one atomic database call is a core database design idea, and this enhancement applies it directly.

Letting the Database Do What It Is Good At

Ordering and searching also move from hand-written algorithms to database operations. Where Enhancement Two's `InOrder()` depended on the BST's own shape to produce sorted output, and `Search()` depended on walking that same shape node by node, Enhancement Three's `PrintAllCourses()` issues a `find({})` query with a sort option on `courseId`, letting MongoDB's query engine, backed by the very index created for uniqueness, produce sorted results without the program maintaining any ordering structure of its own. `DisplayCourseInfo()` and `DeleteCourse()` similarly build a filter document (`{ courseId: <id> }`) and hand it to `find_one()` or `delete_one()`, rather than manually comparing keys down a tree. This reflects an understanding

that once data lives in a database, algorithms like sorting and point lookup are the database's responsibility, and a well-designed client should express what it wants rather than reimplement how to get it.

Schema Design and Bulk-Write Behavior

`courseToDocument()` is where this enhancement makes an explicit schema decision. It converts a `Course` into a BSON document, embedding `preReqs` as a nested array field inside that same document rather than storing prerequisites as separate records in a separate collection. Because a course's prerequisites are always read and written together with the course itself, and the list is small and bounded, embedding is the appropriate document-modeling choice here, and the code makes that choice deliberately rather than defaulting to a relational-style split.

`SeedDbFromCSV()` also shows an understanding of bulk-write behavior that a single-row `Insert()` loop does not. Instead of writing one document per CSV row, it builds a vector of BSON documents and sends them in one `insert_many()` call with `ordered(false)`, which tells MongoDB to continue attempting the remaining inserts even if one row fails, for example, if it collides with the unique index. The result's `inserted_count()` is then compared against the number of rows actually read from the file, so the program can tell the user exactly how many of their courses were saved versus skipped, rather than silently succeeding or aborting partway through a large file.

Summary Comparison

Aspect	Enhancement One / Two (in-memory BST)	Enhancement Three (MongoDB)
Persistence	Course data exists only inside the running process's memory; deleteBST()/unique_ptr cleanup destroys it the moment the program exits, so every session starts empty and requires re-loading the CSV.	Course data is written to a MongoDB collection (snhu_courses.courses) that lives outside the program's process. Data survives program exit and is available to the next run, or to any other client, without re-seeding.
Uniqueness enforcement	Enhancement Two enforces unique courseIds in application code, inside addNode(), by checking for a match before inserting.	CreateUniqueIndex() creates a unique index on courseId directly in the database. The constraint is enforced by MongoDB itself for every write, from any client, not just the ones this program's addNode() happens to intercept.
Insert-or-update logic	The menu handler first calls Search() to check whether a courseId exists, then calls either Insert() or UpdateCourse()	UpsertCourse() performs a single replace_one() call with upsert(true). The database decides atomically, in one

Aspect	Enhancement One / Two (in-memory BST)	Enhancement Three (MongoDB)
	depending on the result -- two round trips and a manual branch in application code.	operation, whether to insert a new document or replace an existing one, and reports back which happened.
Ordering / traversal	InOrder() recursively walks the tree's left and right unique_ptr children to produce alphanumeric order; achieving sorted output requires the tree's own shape to be correct.	PrintAllCourses() issues a find({}) query with a sort option on courseId. Ordering is delegated to the database's query engine rather than to a hand-maintained tree structure.
Bulk load behavior	loadCSV() calls Insert() once per row in a loop; a duplicate row silently overwrites a node with no record of how many rows succeeded.	SeedDbFromCSV() batches every row into a vector of BSON documents and sends them in a single insert_many() call with ordered(false), so one bad or duplicate row does not stop the rest, and the result reports exactly how many of the rows were inserted.
Data modeling	A Course is a plain C++ struct	courseToDocument() explicitly

Aspect	Enhancement One / Two (in-memory BST)	Enhancement Three (MongoDB)
	held in a tree node; its shape is whatever the struct definition says, with no independent schema or validation layer.	maps a Course into a BSON document, embedding preReqs as a nested array within the course document -- a deliberate schema decision appropriate for data that is always read and written together with its parent course.
Failure handling	Errors are mostly structural (null pointers, missing files); there is no concept of a remote data store that can be unreachable.	Every database operation (find, insert, update, delete, index creation, and the initial connection) is wrapped in a try/catch for <code>mongocxx::exception</code>, and a ping command explicitly verifies connectivity to the server before the program proceeds.
Responsibility of the storage class	<code>BinarySearchTree</code> both defines the in-memory structure and implements the traversal algorithms that operate on it.	<code>CourseRepository</code> holds no data structure of its own; it is a thin, well-defined boundary (a repository) between the

Aspect	Enhancement One / Two (in-memory BST)	Enhancement Three (MongoDB)
		application and the database, exposing CRUD operations without exposing how MongoDB stores or indexes the data underneath.

What This Enhancement Demonstrates

Enhancement Three does not just add a database underneath the existing program. It re-derives the program's data-access logic around what a database is actually good at. Uniqueness is enforced by an index instead of an application-level check. Insert-or-update is one atomic operation instead of a manual read-then-branch sequence. Sorted output comes from a query, not from maintaining a self-balancing structure by hand. Bulk loads use a batched, partial-failure-tolerant write instead of a naive per-row loop. Also, the data itself now outlives the process that wrote it, which is the entire point of using a database rather than an in-memory structure in the first place. Taken together, these choices show the `CourseRepository` class is designed the way a data access layer should be, which is as a boundary that exposes clean CRUD operations to the rest of the program while handling indexing, atomicity, and failure modes that only exist once data is persisted outside the application itself.

One trade-off is this version depends on a running MongoDB instance and the `mongocxx/bsoncxx` driver libraries, which the prior, self-contained BST versions did not need.

That dependency is the cost of persistence and is a normal one for a database-backed application, but it does mean Enhancement Three cannot run stand-alone the way Enhancements One and Two could.